

ThreeNative

Lean Product Playbook

A product-development direction for an AI-native game creation platform

THE STRATEGIC THESIS

Direct the game. ThreeNative builds it, proves it, and ships it - without locking away Three.js.

OPEN

ThreeNative Framework

PAID

ThreeNative Studio

LATER

Plugins + Game Marketplace

Prepared for Joao | 27 August 2026

Studio first · Framework open · Ecosystem later

Contents

Executive decision

1. Product definition and category
2. Applying the Lean Product Playbook
3. Target customer
4. Underserved needs and jobs to be done
5. Value proposition and positioning
6. Minimum viable product
7. User experience blueprint
8. Framework product direction
9. Business model and pricing hypotheses
10. Go-to-market
11. Metrics and product-market fit
12. Customer research and validation program
13. Evidence-gated roadmap
14. Plugin ecosystem
15. A curated marketplace for ThreeNative games
16. Risk register
17. Product principles
18. Recommended 90-day program
19. Lean Canvas
20. Decision scorecard
21. Final recommendation

Appendices

- Sample intent contract
- Verification receipt schema
- Research source notes

This is an evidence-oriented working playbook. Market assumptions are separated from product assumptions, and every roadmap phase has a behavioral gate.

Executive decision

The decision: Build **ThreeNative Studio** for AI-first game creators who want to direct an outcome, delegate most implementation, and receive a playable, verified, native build without learning a traditional game engine. Keep the **ThreeNative Framework** open source as the technical foundation, adoption channel, trust layer, and escape hatch. Build plugins only after Studio retention exists. Build a game marketplace only after Studio produces enough quality supply and there is measurable player demand.

ThreeNative should not be positioned as a replacement for Unity, Unreal, or Godot. That comparison forces the product into an impossible editor-and-engine feature war. It should be positioned as an **AI-native game creation platform** whose commercial product is a managed, agentic workflow for planning, building, testing, and shipping games.

The core product promise is:

Direct the game. ThreeNative builds it, proves it, and ships it - without locking away Three.js.

The user is not primarily buying a renderer, a framework, an editor, or an AI chat window. The user is buying **correct delegation**:

- They own the vision, feel, constraints, priorities, and final acceptance.
- The AI owns most implementation decisions, repetitive systems work, testing, optimization, and packaging.
- ThreeNative supplies the runtime, game-oriented abstractions, inspection interfaces, deterministic harnesses, and native platform plumbing that make the delegation trustworthy.
- When the abstraction does not fit, the user or agent drops to ordinary TypeScript and upstream THREE.* instead of hitting a proprietary wall.

This report applies Dan Olsen’s Product-Market Fit Pyramid and six-step Lean Product Process to ThreeNative.¹ It also includes a complementary Lean Canvas. The Lean Canvas is not the central artifact of *The Lean Product Playbook*; it originates from Ash Maurya’s *Running Lean* and is useful here as a one-page business-model hypothesis map.²

Strategy on one page

Decision area	Direction
Primary paid product	ThreeNative Studio
Open-source product	ThreeNative Framework and native runtime interfaces
Initial target	JS-literate AI power users who have already tried to create a 3D game and hit reliability, scale, performance, or packaging limits
Expansion target	Visually driven creators who can direct and judge a game but do not want to code or operate an editor
Job to be done	Turn a visual/gameplay idea into a fun, editable, verified native game without mastering a traditional engine
Core differentiation	Delegation + verification + native portability + raw Three.js escape hatches
Initial wedge	Small, polished, single-player 3D action/adventure vertical slices and short commercial games
Monetization	Studio subscription with included credits and predictable overage
North-star metric	Verified playable iterations accepted per weekly active creator

¹ Dan Olsen, *The Lean Product Playbook* overview and table of contents, <https://dan-olsen.com/book/> (accessed August 27, 2026). See also Wiley, <https://onlinelibrary.wiley.com/doi/book/10.1002/9781119154822>.
² Ash Maurya, *Lean Canvas* and *Running Lean*, <https://ashmaurya.com/> and <https://app.leanstack.com/> (accessed August 27, 2026).

Decision area	Direction
Do not build now	Full visual editor, proprietary scene graph, mandatory ECS, multiplayer platform, console productization, plugin marketplace, game store
Ecosystem sequence	Studio → reusable plugins → publishing services → curated game marketplace

Why this direction is credible

Several external signals support the opportunity, while also showing that the market is moving quickly:

- 1. **AI usage is mainstream, but trust is the bottleneck.** In Stack Overflow’s 2025 survey, 84% of respondents used or planned to use AI tools, yet 46% distrusted output accuracy; 66% cited “almost right” solutions as a major frustration.³ This supports making verification a primary product layer rather than a secondary testing feature.
- 2. **Three.js is an unusually large and familiar foundation.** The current npm page describes it as a JavaScript 3D library and reports roughly 13.8 million weekly downloads and more than 5,000 dependent packages at the captured point in time.⁴ Downloads are not equivalent to unique developers, but they are strong evidence of ecosystem scale, documentation volume, examples, and model familiarity.
- 3. **Prompt-to-game demand is real, but generation alone is not unique.** Rosebud already markets plain-language game creation, editable JavaScript, Three.js-based 3D games, browser publishing, and Windows export.⁵ ThreeNative therefore cannot win on “tell AI to make a game” by itself.
- 4. **Plan-build-test is becoming the expected agent loop.** Roblox announced agentic planning, building, and playtesting inside Studio, and reported that 44% of its top 1,000 creators had used Roblox Assistant or third-party AI tools through MCP during the cited measurement window.⁶ This validates the workflow thesis but also creates urgency: ThreeNative must differentiate through openness, portability, native delivery, and source-level escape hatches.
- 5. **Open core plus paid creation services is already a viable pattern.** GDevelop combines an open-source engine with AI assistance, ready-made behaviors, and paid publishing/export services.⁷ PlayCanvas combines an open-source runtime with a hosted platform and editor.⁸ These examples support the proposed framework/Studio split.
- 6. **Established game professionals are not the best initial audience.** GDC’s 2026 survey reported that 36% of game-industry professionals used generative AI at work, while 52% believed it was having a negative impact on the industry.⁹ This does not mean professionals will never use ThreeNative. It means ThreeNative should not lead with “AI replaces game developers” or spend its first years chasing traditional studios that prioritize manual control, existing pipelines, and specialized roles.

What is known, inferred, and still unproven

Evidence class	What it tells us
Externally observed	AI adoption is high; trust is low. Prompt-to-game products exist. Agentic game creation is accelerating. Open-source core plus paid platform is viable.

³ Stack Overflow, 2025 Developer Survey, AI section, <https://survey.stackoverflow.co/2025/ai> (accessed August 27, 2026). The survey reports 84% using or planning to use AI, 46% distrusting accuracy versus 33% trusting it, 66% frustrated by almost-right solutions, and 45% reporting longer debugging of AI-generated code.

⁴ npm, `three` package page, <https://www.npmjs.com/package/three> (snapshot accessed August 27, 2026). Download and dependent counts are volatile and are not unique-user estimates.

⁵ Rosebud AI Game Creator, <https://rosebud.ai/ai-game-creator> (accessed August 27, 2026).

⁶ Roblox, “Roblox Studio Is Going Agentic,” <https://about.roblox.com/newsroom/2026/04/roblox-studio-going-agentic> (accessed August 27, 2026).

⁷ GDevelop, product and game-maker pages, <https://gdevelop.io/> and <https://gdevelop.io/game-makers> (accessed August 27, 2026).

⁸ PlayCanvas, <https://playcanvas.com/> (accessed August 27, 2026).

⁹ Game Developers Conference, 2026 State of the Game Industry summary, <https://gdconf.com/article/gdc-2026-state-of-the-game-industry-reveals-impact-of-layoffs-generative-ai-and-more/> (accessed August 27, 2026).

Evidence class	What it tells us
Internally demonstrated	ThreeNative already runs upstream three/webgpu without Chromium, owns native JS/runtime plumbing, and has desktop/mobile execution and conformance infrastructure. Internal Android measurements showed a very large V8 advantage over QuickJS on a moving-cube stress case.
Strong strategic inference	Verification, native packaging, reusable systems, and escape hatches form a more defensible wedge than raw generation.
Critical unvalidated hypothesis	Enough creators will surrender implementation control, repeatedly pay for delegated work, and remain satisfied without a traditional scene editor.
Later hypothesis	A plugin economy and curated game marketplace can compound Studio value after creator retention and game supply exist.

The rest of the report turns those claims into a testable product program.

1. Product definition and category

1.1 Canonical definitions

The product language must remain precise because loose terminology creates the wrong roadmap.

Three.js

Three.js is a **3D rendering library**. It provides rendering-oriented capabilities such as a scene graph, cameras, geometry, materials, textures, animation primitives, loaders, and WebGL/WebGPU renderers. It does not, by itself, define a game architecture, game lifecycle, physics system, combat model, navigation system, save system, level editor, multiplayer stack, or publishing workflow.¹⁰

ThreeNative Framework

ThreeNative is a **batteries-included game framework around upstream Three.js**. It retains the real Three.js programming model while adding the runtime and common game systems needed to build larger, native, AI-developed games:

- GPU-first native execution and platform portability
- game lifecycle and normalized platform services
- input, audio, assets, physics integration, characters, cameras, navigation, ragdolls, save/load, and other reusable systems
- deterministic inspection and verification hooks
- packaging and deployment support
- semantic, agent-readable abstractions that reduce repeated code generation and failure surface
- progressive escape hatches down to ordinary TypeScript, raw `THREE.*`, and lower-level WebGPU surfaces where appropriate

ThreeNative abstractions are **accelerators, not capability boundaries**. If a built-in ragdoll, ECS, camera, or physics abstraction is unsuitable, the creator can replace it, install another implementation, or write one directly over Three.js.

¹⁰ npm, `three` package page, <https://www.npmjs.com/package/three> (snapshot accessed August 27, 2026). Download and dependent counts are volatile and are not unique-user estimates.

ThreeNative Studio

ThreeNative Studio is the **commercial AI-native creation environment**. It uses ThreeNative behind the scenes to convert human direction into verified implementation. Its job is not to expose every engine control through a GUI. Its job is to preserve intent, plan work, execute long-running changes, run the game, inspect outcomes, detect regressions, report evidence, and deliver builds.

ThreeNative ecosystem

The longer-term ecosystem has two distinct markets:

1. **Creator infrastructure:** plugins, templates, systems, assets, and agent-capable extensions.
2. **Player distribution:** a curated marketplace for finished games.

They should not be built simultaneously. Creator infrastructure strengthens Studio first; player distribution is a later two-sided marketplace with much higher operational and trust costs.

ThreeNative product architecture

One technical foundation, three business layers.

ThreeNative Studio

Commercial AI workspace

Intent capture · planning · build orchestration · verification · managed native delivery

ThreeNative Framework

Open-source game framework around upstream Three.js

Game systems · agent-readable APIs · inspection · deterministic harness · escape hatches

Native Runtime

GPU-first portability seam

V8/JSC · WebGPU surface · Dawn/wgpu-native · desktop/mobile packaging

Upstream THREE.* remains available at every layer - abstractions accelerate; they do not imprison.

1.2 The category to create

The recommended category is:

AI-directed native game creation

The recommended product descriptor is:

An AI-native game creation platform built on real Three.js.

Avoid these lead categories:

- **"Game engine."** It invites direct comparison with decades of editor, rendering, tooling, asset, platform, and enterprise investment.
- **"Three.js wrapper."** It makes the product sound like packaging glue rather than an autonomous creation system.
- **"No-code game maker."** It obscures source ownership, extensibility, and the advanced-user ceiling.

- **"Vibe-coded game platform."** The term can be useful inside the creator community, but player-facing messaging should not frame the output as disposable or low quality.
- **"Unity killer."** It is strategically unnecessary and not credible.

The strongest category message combines outcome, evidence, and openness:

From reference image to verified native game - with real Three.js code you own.

1.3 The competitive frame

ThreeNative does not need to beat traditional engines feature by feature, but it still competes with them economically: the user chooses where to spend time and whether to start the game at all.

The real alternatives are:

- do not build the game;
- generate a disposable browser prototype with a generic AI agent;
- use Rosebud or another prompt-to-game service;
- use Roblox and accept platform lock-in;
- learn Unity, Unreal, Godot, GDevelop, Construct, or PlayCanvas;
- hire a developer or studio;
- build raw Three.js and manually solve architecture, performance, packaging, and platform issues.

ThreeNative wins only when the target user values **speed to a good outcome, delegated implementation, verification, native delivery, source ownership, and extensibility** more than a mature manual editor.

1.4 Strategic contrast with current products

Product / approach	What it validates	Where ThreeNative must be different
Rosebud AI	Natural-language game creation, editable code, Three.js familiarity, integrated assets, instant browser publishing, and paid AI credits are attractive. ¹¹	Native mobile/desktop runtime, long-horizon verification, scalable game systems, stronger project structure, and no browser-first ceiling.
Roblox Studio	Creators want agentic planning, building, project context, and AI playtesting. ¹²	Open source code, general TypeScript/Three.js ecosystem, non-Roblox distribution, and no platform-specific game model.
GDevelop	Ready-made behaviors, AI help, open source, and paid publishing can coexist. ¹³	AI owns more of the workflow; raw Three.js remains available; verification and native GPU runtime are first-class.
PlayCanvas	An open-source runtime can support a commercial hosted platform and WebGPU-oriented creation. ¹⁴	The primary interaction is delegation rather than a collaborative visual editor, and native shipping is central.
Construct	A large audience values fast, browser-based, low-code game creation; Construct reports more than 250,000 monthly users. ¹⁵	ThreeNative should not recreate event sheets or compete on manual editor polish; it should provide higher autonomy, 3D depth, and source-level freedom.
Traditional engine + generic coding agent	Mature engines provide complete tooling and distribution. Generic agents can modify projects.	ThreeNative is designed around agent legibility, deterministic inspection, reusable semantics, token efficiency, and an integrated proof loop rather than bolting AI onto a human-first editor.

¹¹ Rosebud AI Game Creator, <https://rosebud.ai/ai-game-creator> (accessed August 27, 2026).

¹² Roblox, "Roblox Studio Is Going Agentic," <https://about.roblox.com/newsroom/2026/04/roblox-studio-going-agentic> (accessed August 27, 2026).

¹³ GDevelop, product and game-maker pages, <https://gdevelop.io/> and <https://gdevelop.io/game-makers> (accessed August 27, 2026).

¹⁴ PlayCanvas, <https://playcanvas.com/> (accessed August 27, 2026).

¹⁵ Construct, <https://www.construct.net/en> (accessed August 27, 2026).

Key implication: ThreeNative's moat cannot be "we have AI" or "we use Three.js." It must be the compound system: **native runtime + game framework + project memory + verification harness + agent-readable primitives + open escape hatches + managed delivery.**

2. Applying the Lean Product Playbook

Dan Olsen's model separates the **market** from the **product**. The lower layers of the Product-Market Fit Pyramid define the market hypothesis: target customer and underserved needs. The upper layers define the product hypothesis: value proposition, feature set, and user experience. Product-market fit exists when the upper layers satisfy the lower layers well enough that customers repeatedly choose, use, and pay for the product.¹⁶

ThreeNative Product-Market Fit Pyramid

Adapted from Dan Olsen: market hypotheses below, product hypotheses above.

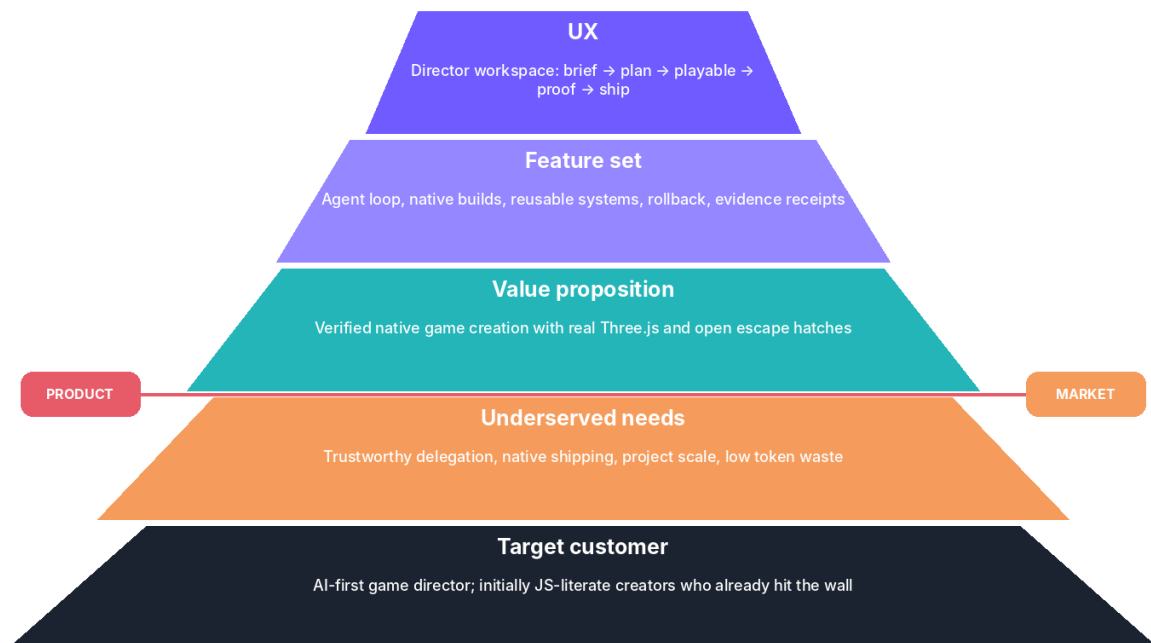


Figure 1. ThreeNative's working Product-Market Fit Pyramid. Every layer is a hypothesis until customer behavior validates it.

The corresponding Lean Product Process is:

1. Determine the target customer.
2. Identify underserved customer needs.
3. Define the value proposition.
4. Specify the MVP feature set.
5. Create an MVP prototype.
6. Test the MVP with customers and iterate.¹⁷

For ThreeNative, the most important discipline is to keep the layers separate. A technically impressive native runtime does not prove that creators want to delegate game implementation. A beautiful Studio interface does not prove that

¹⁶ Dan Olsen, *The Lean Product Playbook* overview and table of contents, <https://dan-olsen.com/book/> (accessed August 27, 2026). See also Wiley, <https://onlinelibrary.wiley.com/doi/book/10.1002/9781119154822>.

¹⁷ Dan Olsen, *The Lean Product Playbook* overview and table of contents, <https://dan-olsen.com/book/> (accessed August 27, 2026). See also Wiley, <https://onlinelibrary.wiley.com/doi/book/10.1002/9781119154822>.

creators will pay for native builds. A marketplace concept does not prove that enough good games will exist. Each claim needs its own evidence.

2.1 Problem space versus solution space

The product should resist premature solution language during customer research. A creator rarely wakes up wanting a “WebGPU-native JavaScript runtime” or an “agent-capable plugin registry.” Those are solutions. The underlying problems are more likely to sound like:

- “AI made a cool demo, but every new change breaks something.”
- “I can describe the game, but I cannot spend a year learning an engine.”
- “The browser version looks fine on my laptop and falls apart on mobile.”
- “I do not know whether the agent actually tested the combat loop.”
- “Packaging for desktop and Android is a second project.”
- “The generated code is too ad hoc to keep extending.”
- “I want to own and modify the result instead of being trapped in a closed creator.”

The initial research program should therefore discuss the creator’s last attempted game, what they did, where it failed, what they abandoned, and what they paid for. It should not begin by pitching ThreeNative features.

2.2 The adaptation used in this report

This report uses four connected artifacts:

Artifact	Purpose
Product-Market Fit Pyramid	Connects the target customer and underserved needs to the value proposition, MVP feature set, and UX.
Needs hypothesis map	Forces an explicit view of which needs are important and poorly served today.
Lean Canvas	Captures business-model assumptions such as channels, revenue, costs, and unfair advantage.
Evidence roadmap	Sequences product work by learning gates rather than by the ambition of the final platform.

The Needs map and Lean Canvas are not substitutes for interviews or usage data. They are founder hypotheses that make disagreement visible and tell the team what to test next.

3. Target customer

3.1 Beachhead segment

The recommended initial target is deliberately narrower than “anyone who wants to make a game”:

JS-literate AI power users who have already attempted a 3D game, can judge whether the result is good, and have hit the limits of disposable AI-generated prototypes, raw Three.js architecture, mobile performance, or native packaging.

This segment is attractive for the first version because its members:

- already understand prompting and iteration;
- can inspect code when the system fails;
- are more likely to appreciate source ownership and raw Three.js escape hatches;
- can give precise technical feedback;
- experience the portability and verification problems directly;

- do not require a complete no-code interface on day one;
- can become early contributors to the open-source framework and plugin vocabulary.

The product can later expand toward non-coding visual creators, but designing the first version for complete beginners would force the team to solve editor ergonomics, asset curation, game design education, safety, and support before the delegation loop is proven.

3.2 Primary persona: the AI-first game director

Working name: Alex, the AI-first game director

Profile:

- Uses ChatGPT, Claude, Codex, or similar tools frequently.
- May be a web developer, product engineer, designer, indie hacker, technical artist, or highly technical hobbyist.
- Has game ideas, references, and taste, but does not want to become a specialist in Unity, Unreal, or Godot.
- Accepts that AI may choose implementation details as long as the direction, constraints, quality bar, and final acceptance remain human-controlled.
- Wants to produce a polished vertical slice, short game, portfolio piece, game-jam entry, experimental commercial title, or interactive experience.
- Values speed and outcome more than manually controlling every transform, component, and node.
- Still wants code ownership and the ability to intervene when needed.

Core statement:

“I want to direct the game, not operate every tool used to make it.”

Success looks like:

- A reference image and concise brief become a playable level quickly.
- The creator can say “combat feels slow” or “make the forest more oppressive” instead of manually editing dozens of objects.
- The agent can work for a meaningful period without supervision.
- Each major change arrives with evidence that it runs, is playable, meets performance budgets, and did not break known flows.
- The creator can install a native build on a target device and can always open the source.

Anxieties:

- The agent will claim success without testing.
- Credits will disappear into failed loops.
- The game will become unmaintainable after the first impressive demo.
- A proprietary platform will trap the project.
- Native export will fail at the last moment.
- AI will erase intentional design choices or make the game generic.

What earns trust:

- visible plans and acceptance criteria;
- checkpoints and one-click rollback;
- deterministic proof receipts;
- explicit uncertainty and known limitations;
- local/source access;
- predictable spend limits;
- successful builds on actual target devices;

- the ability to pin constraints and replace abstractions.

3.3 Secondary personas

The visual creator with technical judgment

This creator can communicate visual and gameplay intent, select references, and judge quality, but writes little or no code. This segment may eventually be larger than the beachhead. It needs stronger onboarding, curated templates, safer autonomy, and a more visual way to express constraints. It should become the expansion segment after the first product loop is reliable.

The rapid-prototype indie team

A one-to-five-person team may use Studio to create vertical slices, explore mechanics, or automate repetitive implementation. This segment values collaboration, version control, review workflows, and handoff to conventional development. It may pay more, but it will also demand stronger determinism, integration, and support.

The framework-first developer

This user adopts the open-source framework without Studio, builds manually or with a generic agent, and contributes issues, examples, and plugins. They are not necessarily the primary paying customer, but they are essential to distribution, credibility, and ecosystem growth.

3.4 Anti-personas

ThreeNative should explicitly decline to optimize its early roadmap for:

- **AAA and large established studios** that require mature console certification, specialized department workflows, deterministic manual control, enterprise support, and years of proven production history.
- **Manual-editor purists** who want to place and tune every object through gizmos and inspector panels and do not want an agent changing anything they did not explicitly select.
- **Non-technical creators demanding a one-prompt finished game** with no willingness to provide direction, evaluate tradeoffs, or iterate.
- **Multiplayer-first founders** whose real problem is live operations, networking, moderation, anti-cheat, and economy design.
- **Users whose primary requirement is pixel-perfect scene editing** rather than delegated outcome quality.

These users may become addressable later, but chasing them now would pull the product into a traditional engine and editor feature war.

3.5 The control bargain

The defining segmentation variable is not coding skill. It is the creator’s willingness to trade **implementation control** for **speed and verified outcomes**.

ThreeNative should test this explicitly. A useful interview and onboarding question is:

“Which decisions must remain yours, and which decisions are you comfortable delegating if the result meets your quality bar?”

The expected boundary is:

Human-owned	AI-owned	Shared
Vision, references, game feel, audience, constraints, priorities, final acceptance	Code structure, system selection, asset plumbing, repetitive implementation, tests, optimization attempts, packaging	Scope, tradeoffs, budget, risky changes, release readiness

The product should allow the boundary to move per project and per task. Delegation is the default; explicit control remains available.

4. Underserved needs and jobs to be done

4.1 Core job to be done

When I have a visual or gameplay idea for a 3D game, help me turn it into a fun, editable, verified native game without requiring me to master a traditional engine or supervise every implementation detail.

This job has four dimensions.

Functional jobs

- Translate a reference, brief, or existing prototype into a playable game.
- Iterate on feel, mechanics, visuals, and content through natural direction.
- Reuse proven game systems instead of generating infrastructure repeatedly.
- Detect regressions and confirm that important flows still work.
- Meet frame-time, memory, input, and packaging constraints on real devices.
- Produce installable builds and source code that remain extensible.

Emotional jobs

- Feel capable of creating a real game rather than merely generating a demo.
- Feel confident that autonomous work did not silently damage the project.
- Avoid the intimidation and guilt of needing to learn a complex engine before starting.
- Preserve creative authorship even when implementation is delegated.

Social jobs

- Share something polished enough that others treat it as a game, not "AI slop."
- Demonstrate taste, direction, and originality.
- Potentially sell the result and build a reputation as a creator.

Switching jobs

- Move beyond a generic coding agent plus browser prototype.
- Avoid restarting in Unity, Unreal, or Godot when the prototype becomes serious.
- Avoid platform-specific packaging projects for Android, desktop, and later iOS.
- Avoid closed prompt-to-game platforms that do not provide an acceptable escape hatch.

4.2 Needs hypothesis map

Founder-hypothesis need map - validate through interviews

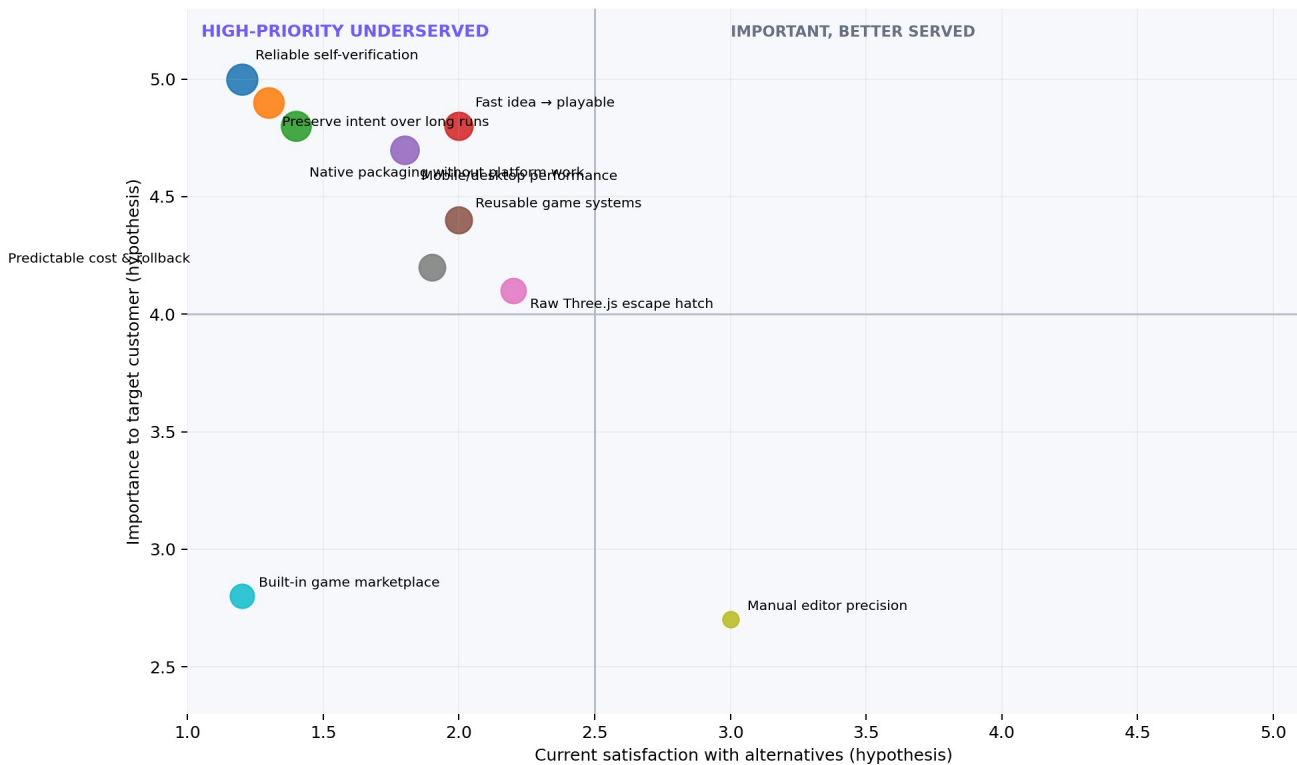


Figure 2. Founder-hypothesis map. Ratings are not customer data; interviews and behavior must replace them.

The highest-priority needs are expected to be:

1. **Trustworthy self-verification.** The system must prove that work functions rather than merely claiming completion.
2. **Preservation of intent over long sessions.** The agent must remember the visual direction, game rules, exclusions, and quality bar.
3. **Native packaging without platform expertise.** The creator should receive an installable artifact, not another integration project.
4. **Predictable change and rollback.** Autonomous work must be checkpointed and reversible.
5. **Good performance on target hardware.** Especially on mobile, the product must move beyond browser/WebView assumptions and measure real devices.
6. **Reusable game systems.** Common capabilities should be installed, configured, and verified rather than regenerated ad hoc.
7. **Raw Three.js escape hatches.** The user must be able to exceed the framework's abstractions without abandoning the project.

A built-in game marketplace is hypothesized to be low-importance for the initial creator job. It becomes valuable only after creators can reliably finish games.

4.3 Need statements for interviews

Rather than asking "Would you use ThreeNative?", test the following need statements against recent behavior:

- I need to know whether an AI-generated change actually works without manually replaying every path.
- I need the agent to remember my design constraints across many changes.
- I need to recover quickly when autonomous work makes the game worse.
- I need to package a working Android or desktop build without learning native toolchains.

- I need reusable systems that do not make the game feel generic or trap me in an opinionated architecture.
- I need to inspect and modify ordinary source code when the high-level workflow is not enough.
- I need predictable costs for a long-running agent.
- I need the output to remain maintainable after the first impressive demo.

For each statement, measure:

- importance;
- current satisfaction;
- frequency of the problem;
- current workaround;
- time and money spent;
- consequence when it fails;
- evidence of an attempted solution.

4.4 The strongest market insight to validate

The central hypothesis is not merely that people want AI to build games. Competitors already validate that interest. The crucial hypothesis is:

A meaningful segment will accept delegated implementation if ThreeNative makes direction persistent, outcomes inspectable, work reversible, and delivery verifiable.

If interviews show that target users still require continuous manual scene manipulation, the product may need a lightweight visual control layer. If they require a traditional editor for most tasks, the differentiation weakens considerably. This is the highest-risk assumption and should be tested before broad framework expansion.

5. Value proposition and positioning

5.1 Value proposition statement

For AI-first creators who want to make real 3D games but do not want to master and operate a traditional game engine, **ThreeNative Studio** is an AI-directed native game creation platform that turns references and direction into playable, verified builds. Unlike generic coding agents, browser-first prompt-to-game tools, or closed game platforms, it combines a native GPU-first runtime, reusable game systems, autonomous verification, and ordinary Three.js source code that remains fully accessible.

5.2 The four-pillar promise

1. Portability

Write and generate against the same upstream Three.js model, then run through a native GPU-first runtime across supported desktop and mobile targets. The user should not become the integration engineer for V8, JSC, Dawn, wgpu-native, Android lifecycle, native input, packaging, or platform file systems.

2. Verifiability

Every significant autonomous change should produce a proof package: what changed, what was run, what passed, what failed, how the output looked, how it performed, and how to reproduce it. Verification is a product surface, not an internal test suite.

3. Reusable game abstractions

Common systems such as character control, input, cameras, physics integration, navigation, ragdolls, save/load, interactions, and projectiles should be stable, semantic building blocks. Their value is not only convenience. They compress tokens, reduce generated code, lower bug surface, and give the agent vocabulary it can inspect and test.

4. Escape hatches

The abstractions must never become a hard box. Raw TypeScript and upstream THREE.* remain available. Creators can bring an ECS, replace physics, write a custom renderer feature, create a bespoke editor, or bypass a built-in system. High-level defaults raise the floor; the underlying renderer prevents a low ceiling.

5.3 Kano-style feature classification

Olsen recommends distinguishing must-haves, performance benefits, and delighters when defining the value proposition.¹⁸

Must-haves

Without these, the product is not trustworthy enough for the target user:

- project persistence and source ownership;
- playable preview;
- checkpoints, diff, and rollback;
- visible task plan and acceptance criteria;
- basic automated play and visual verification;
- stable reusable game primitives;
- one reliable native build target;
- predictable credit/budget controls;
- raw Three.js access;
- clear failure and uncertainty reporting.

Performance benefits

More is continuously better:

- shorter time to first playable;
- longer useful autonomous runs;
- higher verification coverage;
- lower cost per accepted iteration;
- faster native build delivery;
- broader platform support;
- better frame time and memory behavior;
- larger library of reliable game systems;
- fewer rollbacks and manual repairs.

Delighters

These can create memorable differentiation after the must-haves work:

- visual-reference matching with side-by-side evidence;
- an agent that plays and comments on game feel;
- automatic performance regression diagnosis on a real device;
- a proof video generated for each milestone;
- one-click replacement of a system with a plugin or custom implementation;
- automatic creation of a playable store demo and trailer clips;
- collaborative AI roles such as game designer, gameplay engineer, and QA reviewer.

¹⁸ Dan Olsen, *The Lean Product Playbook*, Chapter 5, "Define Your Value Proposition," Wiley, <https://onlinelibrary.wiley.com/doi/10.1002/9781119154822.ch5>.

The team should not spend heavily on delighters while native builds, rollback, or verification remain unreliable.

5.4 Positioning language

Recommended headline

Direct the game. Get a verified native build.

Recommended supporting line

Start from a screenshot, a brief, or an existing Three.js project. ThreeNative Studio plans, builds, plays, tests, and packages the game using real Three.js code you own.

Proof points

- Real upstream three/webgpu, not a proprietary scene model.
- Native GPU runtime across supported platforms.
- Checkpointed autonomous work with replayable evidence.
- Reusable game systems with raw Three.js escape hatches.
- Source code and builds remain the creator's.

Messaging to avoid

- "Make any AAA game from one prompt."
- "No skill required."
- "Replace game developers."
- "The fastest game engine."
- "AI-generated games marketplace."

Those claims attract the wrong expectations, invite hostile comparisons, and damage trust.

6. Minimum viable product

6.1 The narrow outcome

The first paid product should not promise "build any game." It should reliably deliver one constrained outcome:

Create and iterate on a small, polished, single-player 3D action/adventure vertical slice from references and direction, then deliver a verified desktop build and an Android build from the same project.

A suitable vertical slice contains:

- one small level;
- one player controller;
- one core interaction or combat loop;
- a small number of enemy or obstacle types;
- a win/lose or completion condition;
- basic audio and UI;
- a repeatable 5-to-15-minute experience;
- deterministic test flows;
- target performance budgets.

This wedge is rich enough to prove the framework and Studio loop, but small enough to verify. It avoids the compounding complexity of multiplayer, open worlds, live economies, user-generated content, or hundreds of hours of content.

6.2 MVP feature set

A. Intent brief and project contract

The creator uploads one or more visual references and defines:

- fantasy and emotional goal;
- core loop;
- player verbs;
- visual constraints;
- must-have and forbidden elements;
- target devices;
- performance budget;
- scope and credit budget;
- acceptance tests.

The brief becomes a persistent, versioned contract. The agent should cite or update it when making tradeoffs.

B. Plan and autonomy controls

Before implementation, Studio generates a task plan with acceptance criteria, risk, estimated credits, and dependencies. The creator can choose an autonomy mode:

1. **Suggest:** the agent proposes; the creator approves each change.
2. **Guided:** the agent implements small tasks and pauses at milestones.
3. **Delegated:** the agent executes the approved plan within budget and constraints.
4. **Autonomous:** the agent may re-plan, test, optimize, and roll back until the goal or budget limit is reached.

The MVP can ship with Guided and Delegated modes first. Suggest is naturally available through plan review; fully Autonomous mode should wait until rollback and verification are proven.

C. Playable preview and interaction

The user can run the game in an embedded or streamed viewport, provide keyboard/controller/touch input, capture a moment, and attach feedback to the current state. The preview is not a visual scene editor. It is the primary way to judge the outcome.

D. Agent execution and reusable systems

The agent can use:

- upstream Three.js;
- ThreeNative project templates;
- stable character, camera, input, physics, interaction, enemy, health, save, and audio primitives;
- asset import and normalization;
- project-local custom TypeScript;
- a constrained package/plugin catalog.

For the first wedge, the framework should support one excellent gameplay stack rather than several incomplete architectures.

E. Verification receipt

Every milestone returns a structured receipt containing:

- intent and acceptance criteria addressed;
- source diff and files changed;

- unit/integration/conformance tests executed;
- automated play routes executed;
- screenshots or short clips at named checkpoints;
- scene and physics assertions;
- frame-time and memory results on configured targets;
- native build status and artifact hashes;
- known limitations and unverified assumptions;
- exact checkpoint to restore.

The receipt must distinguish **tested**, **observed**, **inferred**, and **not verified**. It must never collapse those into a confident summary.

F. Checkpoints and rollback

Studio creates immutable checkpoints before risky changes and after accepted milestones. The creator can compare the current build with a prior checkpoint, accept, fork, or restore. Rollback should include source, assets, project intent, dependencies, and verification state.

G. Managed native delivery

The MVP should deliver:

- fast development preview;
- a downloadable desktop build for the primary supported desktop target;
- an Android APK or app bundle tested on a defined hardware lane;
- build logs and reproducible metadata.

The exact initial desktop target should follow the team's strongest current build lane. Android is strategically important because it demonstrates the product's native-mobile advantage. iOS can follow once signing, device testing, and support are reliable.

H. Source ownership and export

The user can inspect the complete TypeScript/Three.js project, connect or export to Git, open it locally, replace a system, and continue without Studio. Studio-specific metadata should be additive, documented, and removable where practical.

I. Credit and budget visibility

Before a plan runs, the creator sees:

- estimated credit range;
- hard maximum;
- included verification work;
- events that require approval;
- remaining project and monthly budget.

The user should pay for accepted creative progress, not be forced to understand raw model tokens. Internally, credits can map to model, build, device, asset, and storage costs.

6.3 What the MVP explicitly excludes

- full Unity-style world editor and transform gizmo suite;
- proprietary scene graph or mandatory ECS;
- multiplayer/networking backend;
- open-world streaming;
- consoles as a product promise;

- iOS if signing and real-device verification are not ready;
- team roles and enterprise permissions;
- paid plugin marketplace;
- consumer game marketplace;
- arbitrary model/provider orchestration exposed to the user;
- one-prompt full-game claims;
- automatic publishing to every store;
- procedural generation as a headline feature.

Each exclusion protects the learning goal: prove that users value delegated, verified native game creation enough to return and pay.

6.4 MVP success criteria

The following are proposed product gates, not industry benchmarks. They should be adjusted after the first pilot cohort.

Gate	Initial threshold
Qualified pilots who reach a playable first slice	at least 8 of 10
Median time from completed brief to first playable	under 30 minutes for a template-based slice
Pilots who accept at least one AI-produced milestone	at least 7 of 10
Pilots who install and launch a native build	at least 6 of 10
Accepted milestones with a complete verification receipt	at least 90%
Projects restored successfully from checkpoint after a forced failure	100% in the pilot set
Week-3 active retention among paid pilots	at least 50%
Pilots willing to pay again or continue a subscription	at least 5 of 10

A product can generate impressive demos and still fail these gates. Retention and repeated acceptance matter more than social-media reactions.

7. User experience blueprint

Olsen treats UX as a core product layer because the feature set only creates value when users can understand and successfully use it.¹⁹ ThreeNative’s UX should be designed for **direction, review, and evidence**, not for continuous low-level authoring.

¹⁹ Dan Olsen, *The Lean Product Playbook*, Chapter 8, “Apply the Principles of Great UX Design,” Wiley, <https://onlinelibrary.wiley.com/doi/10.1002/9781119154822.ch8>.

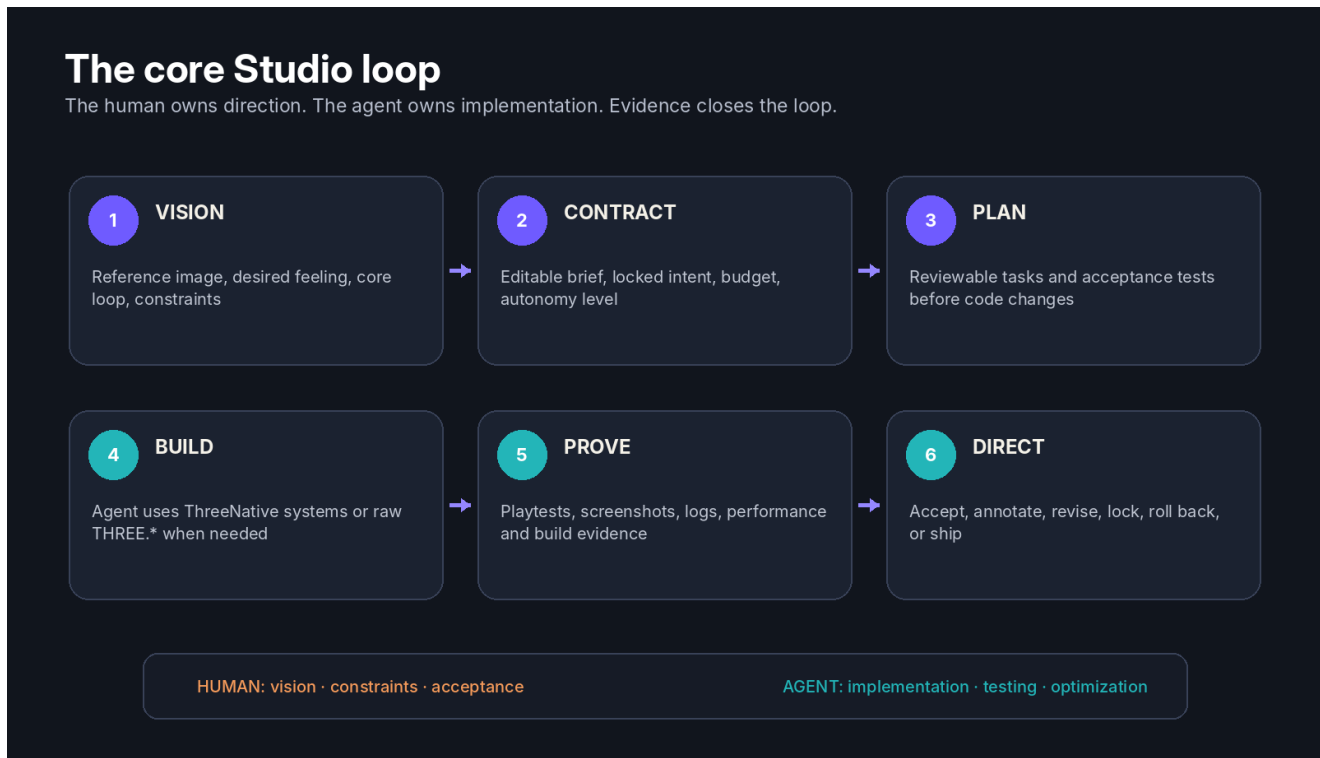


Figure 3. The core Studio loop. The human owns vision and acceptance; the agent owns implementation and proof.

7.1 The interaction model

The Studio should feel like a **direction-and-evidence cockpit** with code and scene details available on demand. It should not be:

- a chat transcript with a hidden build system;
- a clone of Unity's editor;
- a magical one-prompt generator;
- a dashboard full of agent monologue;
- an opaque credit slot machine.

The core loop is:

1. **Vision:** provide reference images, desired feeling, core loop, constraints, and target devices.
2. **Contract:** convert direction into a persistent, editable brief and acceptance tests.
3. **Plan:** review tasks, risk, dependencies, and spend before execution.
4. **Build:** let the agent use ThreeNative systems or ordinary Three.js code.
5. **Prove:** run automated play, screenshots, assertions, performance tests, and native builds.
6. **Direct:** accept, annotate, revise, lock, roll back, fork, or ship.

The user should always know which step the project is in and what evidence is required to advance.

7.2 Recommended workspace

Recommended Studio workspace - not a Unity clone

A direction-and-evidence cockpit, with code and scene details available on demand.

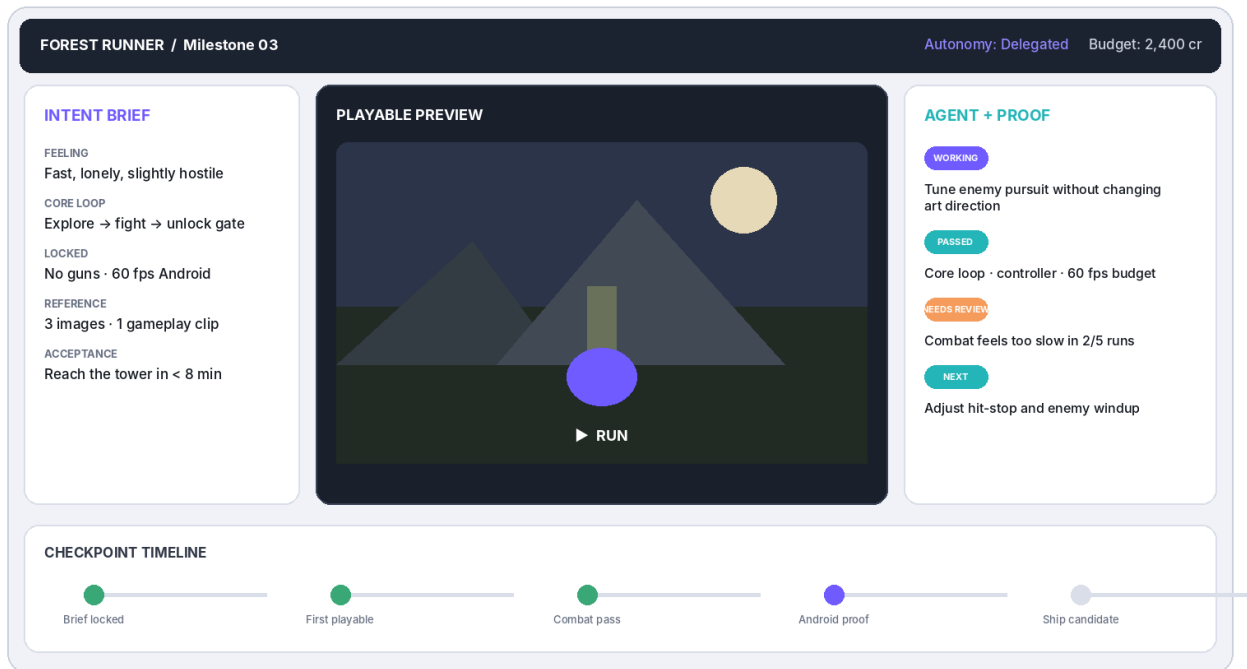


Figure 4. A proposed workspace. It keeps intent, a playable result, evidence, and project history visible without recreating a traditional engine editor.

Persistent intent panel

This panel shows the active creative contract:

- concise game fantasy;
- visual references;
- core loop;
- locked constraints;
- current milestone;
- acceptance criteria;
- target devices and performance budget.

It should be short enough to scan. Long design documents can exist underneath, but the current contract must remain visible.

Playable preview

The center of gravity is the running game. The user can:

- play immediately;
- switch input modes;
- capture a screenshot or clip;
- click a point and attach feedback;
- compare before/after checkpoints;
- view optional overlays for performance, collision, navigation, or scene identity.

A lightweight selection/inspection mode may be useful, but manual placement should not become the main workflow.

Agent and proof panel

The panel should emphasize state and evidence:

- current objective;
- plan step;
- tests running;
- blockers and uncertainties;
- remaining budget;
- latest verification result;
- actions needing human judgment.

Avoid streaming every internal thought. Users need concise, reviewable decisions and evidence.

Checkpoint timeline

A timeline shows accepted milestones, automatic safety checkpoints, experiments, rollbacks, and releases. The user can branch from an earlier checkpoint without destroying the current version.

7.3 Onboarding flow

The first session should produce a meaningful result quickly without pretending that one prompt is enough.

1. Choose a constrained template: third-person exploration/combat, top-down action, or first-person atmospheric exploration.
2. Upload one to five references or start from an existing project.
3. Answer five high-value questions:
 - What should the player repeatedly do?
 - What should the experience feel like?
 - What must be present?
 - What must never happen?
 - Where must it run?
4. Review the generated one-page intent contract.
5. Approve the first-slice plan and budget.
6. Watch the first playable milestone arrive with a verification receipt.
7. Play, annotate, and issue one directional change.
8. Install the first native build.

The activation event is not account creation or prompt submission. It is:

The creator accepts a verified playable change and successfully runs a native build.

7.4 Communicating uncertainty

The product should use explicit status labels:

- **Verified:** a defined test or observation passed.
- **Observed:** the system captured evidence but did not assert correctness.
- **Inferred:** the agent believes the result follows from evidence but did not test it directly.
- **Blocked:** required evidence could not be produced.
- **Not evaluated:** outside the current acceptance scope.

This language is more credible than a green “Done” badge. It also creates training data for future agents and helps users decide where to spend attention.

7.5 Human intervention rules

The agent should pause when:

- requirements conflict;
- the change would exceed the hard budget;
- an irreversible external action is required;
- a license or asset provenance issue appears;
- two options materially alter game identity;
- verification fails repeatedly;
- a required target cannot be tested;
- the agent proposes deleting or replacing a locked system.

The agent should not pause for routine implementation choices that fit the approved contract. Excessive confirmation destroys the value of delegation.

7.6 Control without an editor clone

To support precise direction without building a full editor, offer progressively deeper controls:

1. Natural-language and visual annotation.
2. Semantic constraints: "keep this path open," "never spawn behind the player," "lock the tower silhouette."
3. Inspectable object cards and properties for selected items.
4. Code and raw scene access.
5. Custom tools or user-built editors over the same framework.

This creates a high floor without turning manual control into the product's center.

8. Framework product direction

8.1 The framework's job

ThreeNative Framework should make AI-generated Three.js games **durable**. It does not need to model every game genre. It needs to provide stable runtime capabilities, excellent common systems, and enough semantic structure that an agent can reason about the project.

A useful prioritization test for every framework feature is:

Does this capability reduce repeated generated code, improve verification, eliminate platform-specific waste, or create a stable semantic primitive that many games need?

If not, it may belong in an example, plugin, or project rather than core.

8.2 AI-legible abstractions

APIs should optimize for semantic clarity, predictable composition, observability, and testability. A good high-level abstraction should provide:

- a clear purpose and vocabulary;
- typed configuration;
- stable lifecycle;
- inspectable state;
- deterministic events;
- explicit failure modes;

- built-in debug visualization;
- verification hooks;
- a documented path to override or replace it.

For example, a character system should expose grounded state, current locomotion mode, velocity, collision contacts, input intent, and transition events. The agent should not need to infer these from arbitrary user code or visual pixels.

8.3 Token compression as a product metric

Reusable systems reduce more than development time. They reduce:

- prompt and output tokens;
- generated lines of code;
- number of APIs invented per project;
- defect opportunities;
- verification complexity;
- context the agent must reload;
- migration burden across platforms.

For each major abstraction, measure:

- median generated-code reduction versus a raw implementation;
- agent success rate on first integration;
- test coverage reused across projects;
- platform compatibility;
- frequency of overrides;
- defects per accepted project milestone.

This turns “batteries included” into a measurable AI-product advantage.

8.4 What belongs in core

The core should be opinionated where platform complexity creates no creative differentiation:

- app and frame lifecycle;
- native surface and GPU initialization;
- asset loading and storage;
- input normalization;
- audio/device plumbing;
- packaging and build metadata;
- deterministic clocks and seeded execution;
- screenshots, video, telemetry, and inspection;
- test harnesses and automation interfaces;
- project manifests and capability discovery.

The core should offer defaults but remain less opinionated about:

- ECS versus object-oriented organization;
- combat architecture;
- quest structure;
- world composition;
- state-management style;

- animation-state design;
- game-specific data models.

Common solutions can exist as official modules without becoming mandatory architecture.

8.5 The escape-hatch contract

Every official abstraction should document:

- which raw Three.js objects it owns;
- which lifecycle hooks it uses;
- how to inspect its state;
- how to replace one subsystem;
- how to coexist with project-local code;
- whether it can be removed without rewriting the scene;
- which platform capabilities it requires.

The ideal progression is:

```
Studio intent
-> ThreeNative high-level system
-> ThreeNative lower-level primitive
-> ordinary TypeScript + upstream THREE.*
-> raw WebGPU/native capability where supported
```

The user should never discover that choosing Studio permanently moved the game into an undocumented proprietary model.

8.6 Open-source boundary

The recommended split is:

Open source

- ThreeNative Framework;
- runtime interfaces and platform adapters that can legally be public;
- core game-system packages;
- project format and manifests;
- verification protocol and local runner;
- plugin SDK and compatibility tests;
- starter templates and representative examples.

Commercial Studio

- hosted model orchestration;
- long-running agent execution;
- managed build/device farm;
- cloud project memory and artifact storage;
- proof aggregation and review experience;
- asset-generation workflow;
- collaboration, billing, and support;
- hosted plugin distribution and later publishing services.

This boundary makes the framework trustworthy and adoptable while monetizing the expensive managed workflow. It also allows users to leave Studio without losing the game.

9. Business model and pricing hypotheses

9.1 What customers pay for

Customers should not perceive that they are paying for access to Three.js or to a closed engine. They pay for:

- autonomous implementation time;
- verified outcomes;
- reusable systems and managed integration;
- native builds and device testing;
- hosted project memory and artifacts;
- reduced time to a playable result;
- avoidance of platform/toolchain expertise;
- confidence that the project can continue growing.

9.2 Recommended pricing structure

Use a subscription with included credits and predictable overage:

Layer	Purpose
Open Framework	Free local development, community adoption, examples, and escape hatch.
Creator subscription	Studio workspace, monthly credits, project memory, verification, standard native builds, and artifact storage.
Pro subscription	More autonomy, device lanes, parallel work, larger projects, advanced verification, and priority builds.
Usage overage	Additional verified work, build/device time, premium models, and asset generation.
Teams later	Shared projects, roles, audit history, private plugin catalogs, and support.

Avoid exposing raw token billing as the primary mental model. Credits should map to understandable work units such as "verified implementation," "native validation run," or "asset generation," while the interface shows a maximum cost before execution.

9.3 Pricing experiments

Do not finalize price from competitor pages. Start with willingness-to-pay evidence:

1. Recruit ten qualified founding creators with real projects.
2. Charge for the pilot; free users produce weak price evidence.
3. Test three value frames:
 - subscription for ongoing creation;
 - milestone package for a verified vertical slice;
 - subscription plus metered native/device work.
4. Ask what they would replace: freelance work, coding-agent spend, engine learning time, or an abandoned project.
5. Track cost per accepted milestone and gross margin, not merely credits consumed.

A reasonable early test range is **USD 39 to 149 per month** for self-serve creator plans, with a higher-priced concierge pilot. These are hypotheses, not recommended final prices. The correct price depends on accepted output, model cost, build/device cost, and retention.

9.4 Revenue expansion

Only after Studio retention:

- paid plugins and templates;
- premium asset packs or generation workflows;
- managed publishing and store submissions;
- revenue share from the creator marketplace;
- optional hosting, multiplayer services, analytics, or cloud saves;
- higher-tier device and platform certification services.

The business should not depend on marketplace revenue before it has a healthy creation product.

9.5 Cost model

The major variable costs are likely to include:

- model inference and agent runtime;
- cloud build compute;
- real-device testing;
- artifact storage and bandwidth;
- asset generation and processing;
- support for failed native builds;
- moderation and provenance review later.

Track **cost per accepted verified iteration**, not cost per prompt. A cheap generation that fails verification or is rejected is waste. A more expensive run that produces an accepted milestone may have better unit economics.

10. Go-to-market

10.1 The initial story

The best demonstration is not a benchmark or architecture diagram. It is an end-to-end transformation:

```
reference image + five-minute brief
-> visible plan and budget
-> autonomous implementation
-> agent playtest and proof receipt
-> creator-directed revision
-> native Android/desktop build
-> complete Three.js source
```

The marketing message should be **show, prove, ship**.

10.2 Acquisition channels

Open-source framework

- high-quality GitHub repository and documentation;
- runnable examples that agents can understand;
- direct compatibility with upstream Three.js patterns;
- migration guides from browser-first Three.js projects;
- transparent benchmarks and conformance results;

- plugin and system examples.

Demonstration content

- time-lapse builds with checkpoints and failures included;
- "reference to native APK" case studies;
- side-by-side performance and device results;
- proof receipts attached to public demos;
- short breakdowns of how reusable systems compressed the work;
- creator stories focused on direction and outcome, not "AI made everything."

Community

- Three.js and React Three Fiber communities;
- AI coding and indie-hacker communities;
- game jams designed around verified native builds;
- Discord/community workspace for early creators;
- partnerships with asset creators and technical educators.

Existing-project wedge

A powerful acquisition path may be:

Bring an existing Three.js game or AI-generated prototype. Studio makes it durable, verifies it, and ships it natively.

This starts from demonstrated pain and avoids asking the user to trust an empty canvas.

10.3 Launch cohorts

Cohort 1: Founding creators

Ten highly engaged, technically capable creators. Founder-led onboarding, weekly interviews, manual support, paid participation, narrow templates.

Cohort 2: Invite-only Studio

Fifty to one hundred creators. Productized onboarding, stable credit model, two native targets, reusable proof receipts, instrumented retention.

Cohort 3: Public creator launch

Only after repeated project retention, predictable native delivery, acceptable gross margins, and clear support boundaries.

10.4 Brand posture

ThreeNative should be confident but not antagonistic:

- not "traditional game development is dead";
- not "anyone can make GTA in ten minutes";
- not "AI replaces studios";
- not "Three.js was bad."

A stronger posture is:

Game creation is becoming directional. ThreeNative gives agents a native, testable, open foundation while creators retain the vision and the code.

11. Metrics and product-market fit

11.1 North-star metric

Verified playable iterations accepted per weekly active creator

This metric captures the product's core value better than prompts, tokens, files changed, or time spent. An iteration counts only when:

- the game runs;
- required verification is complete;
- the creator accepts the result;
- the checkpoint remains part of the active project lineage.

11.2 Metric tree

Acquisition

- qualified creator sign-ups;
- percentage arriving with an existing prototype or reference;
- open-source-to-Studio conversion;
- cost per qualified creator;
- referral and public-project attribution.

Activation

- brief completion rate;
- time to approved contract;
- time to first playable;
- percentage accepting first milestone;
- percentage launching first native build;
- first-session credit efficiency.

Retention

- weekly active creators;
- projects with accepted iterations in weeks 2, 4, and 8;
- creator return after native build;
- number of distinct milestones per retained project;
- percentage of projects surviving beyond the initial demo.

Quality and trust

- verification pass rate;
- false-positive verification incidents;
- rollback rate;
- creator rejection rate and reason;
- native build success on first attempt;
- performance-regression escape rate;
- percentage of agent claims backed by evidence;
- unresolved uncertainty per milestone.

Economics

- model/build/device cost per accepted iteration;

- gross margin by plan;
- paid credit utilization;
- overage conversion;
- support time per active creator;
- refund and abandoned-run rate.

Framework health

- successful installs;
- active projects;
- example completion;
- external contributors;
- plugin usage;
- raw Three.js escape-hatch frequency;
- API churn and migration burden.

11.3 Product-market-fit survey

Once there are enough retained, representative users, use the Sean Ellis question:

“How would you feel if you could no longer use ThreeNative Studio?”

The commonly used heuristic is that approximately 40% answering “very disappointed” signals strong product-market fit, but the score should be interpreted with cohort quality and behavior, not used as a magic threshold.²⁰

Segment the results by:

- creator technical level;
- project type;
- existing project versus new project;
- native target;
- level of delegated autonomy;
- paid plan;
- number of accepted milestones.

The explanation to “What is the main benefit?” should converge around the intended value proposition. If users say only “it generates things fast,” the moat is weak. Stronger answers would mention trust, completion, native delivery, continuity, or avoiding engine/toolchain learning.

11.4 A quality counter-metric

Every growth metric needs a guardrail:

Percentage of accepted milestones later reverted for quality or correctness reasons.

A rising north-star metric alongside rising quality reverts would indicate that Studio is manufacturing apparent progress rather than durable progress.

²⁰ Sean Ellis Product-Market Fit Survey, <https://pmfsurvey.com/>; see also First Round Review, “How Superhuman Built an Engine to Find Product-Market Fit,” <https://review.firstround.com/how-superhuman-built-an-engine-to-find-product-market-fit/> (accessed August 27, 2026).

12. Customer research and validation program

12.1 Research sequence

Stage 1: Problem interviews

Conduct 15 to 20 interviews with people who attempted a 3D game in the previous 12 months. Require evidence of the attempt: repository, prototype, screenshots, spend, or abandoned plan.

Goals:

- identify real failure points;
- quantify current workarounds and spend;
- understand the control bargain;
- learn which target platforms matter;
- identify the moment prototypes become disposable;
- recruit concierge pilots.

Stage 2: Solution interviews

Show a clickable Studio workflow and a sample proof receipt. Do not lead with the full vision. Test comprehension, trust, desired control, and willingness to hand over a real project.

Qualitative usability studies can be productive with small iterative groups; Nielsen Norman Group commonly works with small studies in the five-to-eight-participant range for focused qualitative research.²¹

Stage 3: Paid concierge pilot

Use the real framework and agents, but allow substantial manual orchestration behind the scenes. The goal is to learn the product contract before automating everything.

The pilot should expose the intended interface:

- intent brief;
- plan and budget;
- playable checkpoints;
- proof receipts;
- native builds;
- rollback.

Manual work behind the interface is acceptable as long as it is tracked. The team must distinguish repeatable product capability from founder heroics.

Stage 4: Instrumented self-serve cohort

Only after concierge users repeat the workflow and pay again. Replace manual steps with productized orchestration and measure activation, retention, cost, and quality.

12.2 Interview screener

Recruit participants who answer yes to at least three:

- Used an AI coding tool weekly in the last three months.
- Attempted a 3D game or interactive experience in the last year.
- Used Three.js, React Three Fiber, Unity, Godot, Unreal, Roblox, GDevelop, Construct, or a prompt-to-game tool.

²¹ Nielsen Norman Group, User Testing, <https://www.nngroup.com/consulting/user-testing/> (accessed August 27, 2026).

- Tried to package a game for desktop or mobile.
- Abandoned or restarted a prototype because it became fragile.
- Paid for AI, assets, freelancers, game tools, or publishing.
- Has a current idea they want to turn into a playable slice.

Exclude participants who have no current or recent creation behavior and are responding only to a hypothetical idea.

12.3 Problem interview questions

1. Tell me about the last 3D game you tried to make.
2. What triggered the project, and what outcome did you want?
3. Show me how far you got.
4. Which parts were surprisingly easy?
5. Where did progress slow or stop?
6. What did AI do well, and what did you stop trusting it to do?
7. How did you determine whether a change actually worked?
8. Did you try to run it on another device or package it? What happened?
9. Which implementation decisions did you care about personally?
10. Which decisions would you happily delegate?
11. What did you spend in tools, credits, assets, or outside help?
12. What would make you return to the project this month?
13. What would prevent you from giving a tool access to the project?
14. What would you need to see before paying for autonomous work?

Do not ask "Would you use this?" Ask for commitments: a project, time, payment, data access, or a scheduled pilot.

12.4 Critical experiments

Hypothesis	Cheapest credible experiment	Passing evidence
Creators will delegate implementation	Give users an editable plan with three implementation choices hidden and ask them to approve direction rather than mechanics	Most qualified users approve delegated execution and do not demand continuous low-level control
Verification materially increases trust	Show identical changes with and without a proof receipt	Users choose the evidenced result, understand the receipt, and reduce manual rechecking
Native delivery is a paid benefit	Offer browser-only and verified Android/desktop outcomes at different prices	Users select or pay more for native delivery, not merely praise it
Reusable systems improve outcomes	Build the same feature with an official system and generated project-local code	Official system reaches acceptance faster, with fewer failures and lower cost
Source escape hatches reduce lock-in anxiety	Show source export and system replacement during a session	Users cite ownership/escape as a reason to proceed or pay
The narrow game wedge is attractive	Recruit against three template concepts	One template produces materially higher completion, return, and payment behavior
Credits can feel predictable	Show pre-run estimate, hard cap, and post-run receipt	Low surprise, low refund demand, and users can predict what another iteration will cost

12.5 Evidence hierarchy

From weakest to strongest:

1. Social-media enthusiasm.

2. Email waitlist.
3. Interview statement.
4. Prototype usage.
5. User provides a real project.
6. User invests repeated time.
7. User pays.
8. User returns and pays again.
9. User ships or sells a game.
10. User refers another qualified creator.

Roadmap decisions should be based on the strongest available evidence, not the loudest reaction.

13. Evidence-gated roadmap

Sequence the company around evidence, not ambition

Each phase earns the right to create the next layer.

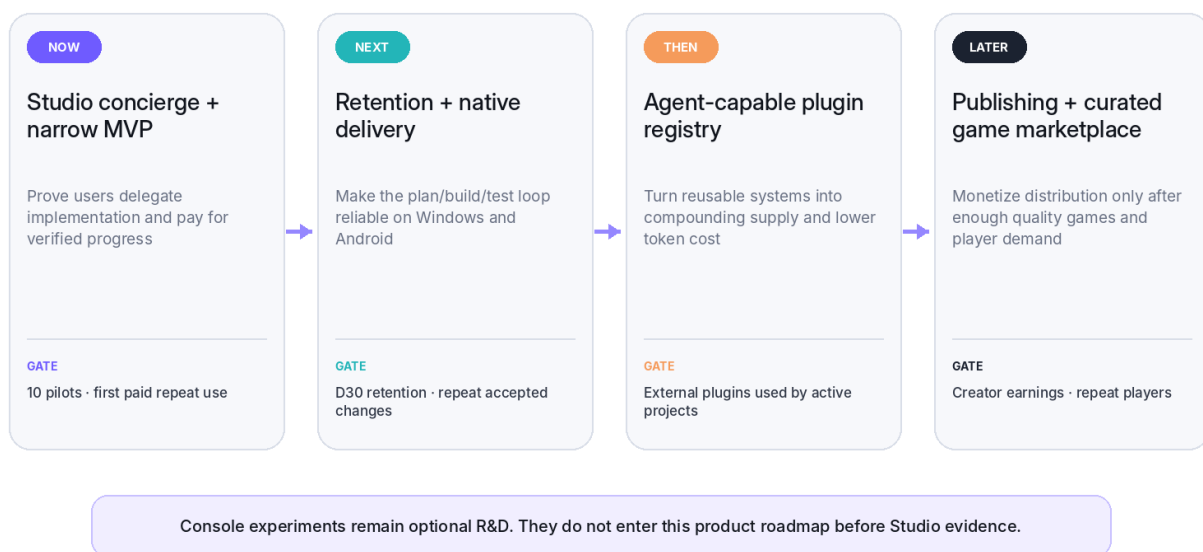


Figure 5. Each phase earns the right to create the next layer. The roadmap advances through evidence gates, not calendar optimism.

Phase 0: Product discovery and founder concierge

Goal: Validate the customer, need priority, control bargain, and paid outcome.

Deliverables:

- 15 to 20 problem interviews;
- one narrow template and sample project;
- clickable Studio workflow;
- proof-receipt schema;
- five paid concierge pilots;
- baseline cost per accepted milestone.

Gate: At least three of five pilots complete a second paid or committed milestone, and verification/native delivery rank among the reasons they continue.

Phase 1: Studio core loop

Goal: Productize intent, plan, build, prove, direct, and rollback.

Deliverables:

- persistent intent contract;
- Guided and Delegated autonomy;
- playable preview;
- checkpoint timeline;
- reusable first-wedge systems;
- proof receipt;
- source export;
- primary desktop and Android build lanes;
- billing and hard credit limits.

Gate: Ten paid creators meet the MVP success criteria, with week-3 retention and acceptable cost per accepted iteration.

Phase 2: Retention and native delivery

Goal: Make multi-week project continuation reliable.

Deliverables:

- performance-regression testing;
- better asset pipeline;
- device matrix and build reliability;
- project memory and locked constraints;
- richer playtest routes;
- migration from existing Three.js projects;
- improved debugging and repair loops;
- iOS only if the build/test lane can be supported honestly.

Gate: Retained creators complete repeated milestones, native builds succeed predictably, and support cost does not overwhelm gross margin.

Phase 3: Agent-capable plugin registry

Goal: Expand capability without forcing the core team to implement every system.

Deliverables:

- plugin manifest and SDK;
- deterministic compatibility tests;
- verification hooks;
- curated official plugins;
- third-party registry;
- installation and update flows;
- project-level dependency locks.

Gate: External plugins are used successfully in active paid projects, and at least several external maintainers continue updating them.

Phase 4: Publishing services

Goal: Help retained creators finish and distribute games.

Deliverables:

- release checklist;
- store artifact generation;
- provenance and license checks;
- crash reporting and basic analytics;
- build signing guidance or managed workflows;
- landing/store page assets;
- demo packaging.

Gate: A meaningful number of creators ship outside the ThreeNative ecosystem and ask for integrated distribution.

Phase 5: Curated game marketplace

Goal: Create player demand and creator revenue only after quality supply exists.

Gate before building:

- a stable cohort of retained creators;
- dozens of finished, quality-reviewed games;
- repeat player behavior in small distribution tests;
- sustainable moderation and support capability;
- clear provenance, refund, safety, and revenue-share policies.

Console experimentation may remain an optional research and marketing lane, but it should not displace Studio evidence work.

14. Plugin ecosystem

14.1 Why plugins matter

The framework promise is “batteries included, escape hatch included.” A plugin ecosystem allows ThreeNative to cover more genres and systems without placing every feature in core. It can also become a creator-economy product, but only if plugins are trustworthy enough for agents to select and integrate autonomously.

The goal is not a conventional package directory with screenshots and vague README files. It is an **agent-capable capability market**.

14.2 Agent-capable plugin contract

Every plugin should declare:

- semantic capabilities and intended jobs;
- typed API and configuration schema;
- supported ThreeNative, Three.js, and platform versions;
- required native capabilities;
- performance and memory expectations;

- deterministic examples;
- verification hooks and acceptance tests;
- debug/inspection surfaces;
- migration and uninstall behavior;
- license, asset provenance, and data handling;
- known incompatibilities;
- whether it modifies project code or remains a dependency.

Studio should be able to answer:

"Can this plugin satisfy the requested feature on Android, how do I verify it, what will it cost, and how do I remove it?"

14.3 Plugin categories

- gameplay systems: inventory, dialogue, quests, combat, abilities;
- locomotion and controllers;
- physics and destruction;
- AI and navigation;
- procedural generation;
- UI and accessibility;
- audio and music systems;
- save, analytics, cloud, and online services;
- asset pipelines and importers;
- verification and playtesting tools;
- visual effects and rendering extensions;
- templates and genre kits.

14.4 Sequencing the plugin business

Stage 1: Official modules

Build a small set of excellent, free, first-party systems that prove the contract.

Stage 2: Curated public registry

Allow third-party plugins, but keep payments out initially. Learn compatibility, moderation, dependency, and update problems.

Stage 3: Paid marketplace

Add payments only when active Studio projects already discover and use plugins. Unity's Asset Store demonstrates that a large creation ecosystem can support paid third-party assets and tools; its publisher page reports 1.7 million monthly users and more than 12,000 active publishers.²²

A creator-friendly revenue share in the **75% to 90% publisher range** is worth testing. The exact split should reflect payment, discovery, verification, support, refunds, and hosting value rather than copying an incumbent.

14.5 Plugin quality score

Rank plugins using behavior, not only reviews:

- successful integration rate;

²² Unity Asset Store, "Start publishing," <https://assetstore.unity.com/publishing/publish-and-sell-assets> (accessed August 27, 2026).

- verification pass rate;
- active-project retention;
- crash and rollback rate;
- platform coverage;
- maintenance recency;
- agent comprehension and configuration success;
- support response;
- uninstall/migration reliability.

Studio can prefer plugins that repeatedly produce accepted outcomes.

15. A curated marketplace for ThreeNative games

15.1 The opportunity

A later marketplace could close the creator loop:

Studio lowers the cost of making games
 -> more creators finish games
 -> quality systems and plugins improve output
 -> publishing becomes easier
 -> players discover and buy games
 -> creator revenue funds more creation

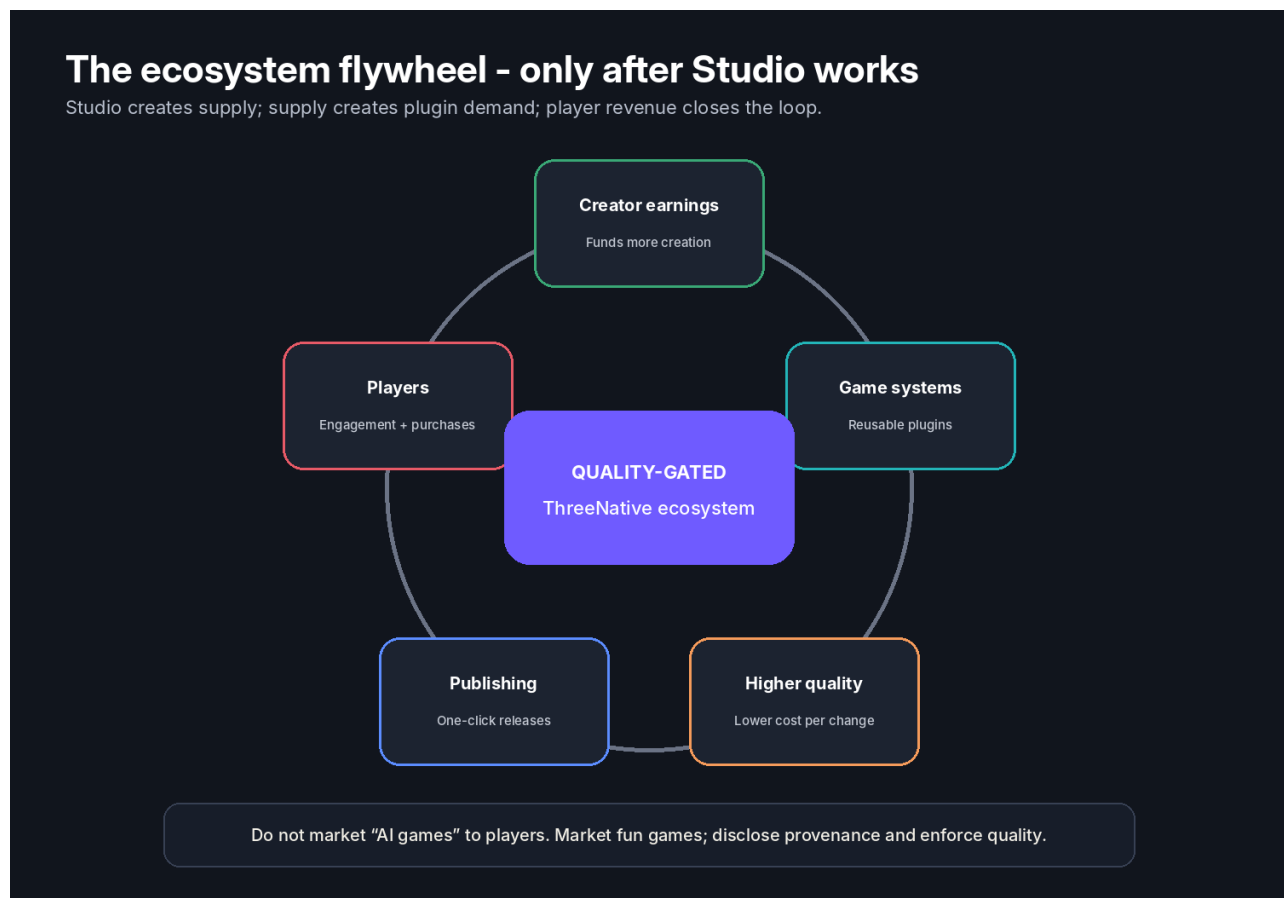


Figure 6. The ecosystem flywheel only activates after Studio reliably creates quality supply.

This is strategically exciting, but it is a separate product with two customers, payments, discovery, moderation, refunds, fraud, safety, intellectual-property disputes, player support, and reputation risk. It should not be treated as a menu item inside the first Studio roadmap.

15.2 Player-facing positioning

Do not sell “vibe-coded games” as the player value proposition. Players primarily care whether a game is fun, polished, safe, compatible, and worth the price.

Recommended framing:

A curated home for inventive, focused games from independent creators.

AI provenance should be disclosed clearly where required or valuable, but quality should lead the marketing.

15.3 Quality gates

A game should not enter the curated catalog merely because Studio generated a build. Require:

- automated crash and launch checks;
- target-device performance thresholds;
- basic input/accessibility requirements;
- complete store assets and disclosure;
- license and asset provenance;
- malware and permission review;
- playable review by a human or trusted curator;
- refund/contact policy;
- evidence that the game offers more than a template reskin.

A lower-friction experimental area can exist separately, but it must not contaminate the main catalog’s trust.

15.4 Listing fee and revenue share hypotheses

Steam Direct charges USD 100 per product and recoups the fee after the product reaches USD 1,000 in adjusted gross revenue.²³ That design provides a useful reference: a small fee can deter spam without permanently taxing legitimate creators.

ThreeNative could test a **recoupable quality bond** rather than a simple listing fee. For example:

- creator deposits a modest amount;
- the game passes automated and human quality review;
- the amount is returned or credited after an initial sales/engagement threshold;
- rejected submissions receive actionable evidence and can resubmit.

The platform take should initially be creator-friendly and justified by discovery, payment, hosting, updates, moderation, analytics, and refunds. itch.io demonstrates an alternative open-market model where sellers can configure the platform revenue share.²⁴

15.5 Marketplace launch gate

Do not build a consumer storefront until all are true:

- at least 30 to 50 creators have retained multi-milestone projects;
- at least 25 quality-reviewed games are genuinely ready for players;
- external distribution tests show repeat play or purchasing;
- at least several creators have earned money outside the marketplace;
- moderation, provenance, and refund processes are staffed;

²³ Valve, Steamworks Partner Program / Steam Direct, <https://partner.steamgames.com/steamdirect> (accessed August 27, 2026).

²⁴ itch.io, About and creator revenue sharing, <https://itch.io/docs/general/about> (accessed August 27, 2026).

- Studio itself has healthy retention and unit economics.

Before that point, use showcases, game jams, bundles, and limited curated drops to test player demand cheaply.

16. Risk register

Risk	Why it matters	Early mitigation
Generation becomes commoditized	Foundation models and competitors will all create code and assets.	Build around verification, native delivery, project continuity, semantic systems, and source openness.
AI-slop reputation	Low-quality output can damage creator and player trust.	Narrow templates, human acceptance, quality gates, provenance, and outcome-first branding.
Verification gives false confidence	A green receipt that misses real defects is worse than no receipt.	Explicit coverage/uncertainty labels, diverse tests, device evidence, failure injection, and audit trails.
Users do not surrender enough control	The product may need an editor it cannot afford to build.	Test the control bargain early; add lightweight semantic/visual controls only where behavior demands them.
Framework scope explodes	Trying to include every game system recreates an engine company.	Core criteria, plugins, one-wedge focus, and raw Three.js escape hatches.
Native platform support consumes the roadmap	Packaging and device variance can overwhelm the Studio team.	Support a declared matrix, automate evidence, fail closed, and add platforms only after reliability gates.
Agent cost destroys gross margin	Long autonomous loops may be expensive and unpredictable.	Hard budgets, task-level estimates, cheaper verification models, caching, reusable systems, and cost per accepted iteration.
Model/provider dependence	Pricing, quality, or policy changes can disrupt the product.	Internal model abstraction, multi-provider evaluation, reproducible tasks, and local execution where practical.
Open-source value leaks without conversion	Framework adoption may not translate into Studio revenue.	Design Studio around managed capabilities that are expensive to self-host, while preserving genuine open-source value.
Lock-in perception	Creators may distrust a proprietary AI platform.	Source export, documented project format, local runner, dependency manifests, and raw Three.js compatibility.
Plugin supply-chain risk	Plugins can introduce breakage, malware, licensing, or abandonment.	Signed packages, capability permissions, compatibility tests, provenance, sandboxing where possible, and maintenance scores.
Marketplace moderation and IP	AI-assisted assets increase provenance and infringement disputes.	Delay marketplace, require source/provenance records, establish takedown and refund policies, and obtain specialist legal review.
Traditional engine vendors move quickly	Roblox and others are adding agentic workflows.	Own the open Three.js/native/escape-hatch position and move faster in a focused segment.
The output is fun only by accident	Technical correctness does not create enjoyable games.	Include game-feel acceptance, human play review, template expertise, and player testing rather than claiming automation solves design.

17. Product principles

1. **The creator directs; the agent implements.** Human vision, constraints, and acceptance remain explicit.

2. **Evidence over confidence.** No important “done” state without a reproducible proof receipt.
3. **Real Three.js remains accessible.** Abstractions accelerate; they do not imprison.
4. **Native delivery is part of the outcome.** A web preview is not equivalent to a tested device build.
5. **Durable progress beats impressive generation.** Optimize for accepted milestones that survive future changes.
6. **The playable game is the center of the UX.** Chat, plans, code, and metrics serve the result.
7. **Spend is visible and bounded.** Autonomy without budget control is not trustworthy.
8. **Framework features earn their place.** Prioritize platform waste, reusable semantics, verification, and token compression.
9. **Do not build a Unity clone.** Add precise controls where needed without making manual operation the product.
10. **Earn the ecosystem.** Studio retention comes before plugins; quality supply comes before a game marketplace.
11. **Market games on quality, not generation method.** Creator provenance can be transparent without becoming the player promise.
12. **Advance through evidence gates.** Public excitement does not replace repeated usage and payment.

18. Recommended 90-day program

Days 1-15: sharpen the problem and recruit

- Conduct the first ten problem interviews.
- Publish a concise category page with two demonstrations: existing Three.js prototype rescue and reference-to-native slice.
- Create the one-page intent-contract template.
- Create the proof-receipt schema and a sample receipt from an existing ThreeNative demo.
- Select one gameplay wedge and one Android reference device.
- Recruit five paid founding creators with real projects.

Decision at day 15: Continue only if creators show strong recent pain, provide real projects, and accept the delegation model.

Days 16-45: run the concierge Studio

- Build a thin interface for contract, plan, playable checkpoints, proof, and rollback.
- Orchestrate agent work manually where necessary.
- Deliver one accepted playable milestone per pilot.
- Deliver at least one native build per viable project.
- Track every manual intervention, cost, failed verification, and user correction.
- Interview each creator immediately after acceptance or rejection.

Decision at day 45: Identify which parts of the loop are repeatable enough to productize and which parts depend on founder expertise.

Days 46-75: productize the narrow loop

- Automate checkpoint creation and restoration.
- Implement the structured proof receipt.
- Stabilize the first reusable gameplay systems.
- Add budget estimates and hard caps.
- Productize desktop/Android artifact delivery.
- Add instrumentation for activation, acceptance, rejection, cost, and retention.

- Run a second paid cohort with less founder intervention.

Decision at day 75: Confirm whether retained users value verification/native delivery or only generation speed.

Days 76-90: choose the next bet

Based on evidence, choose one:

- improve retention and project continuity;
- deepen Android/native reliability;
- improve the intent/control UX;
- strengthen reusable game systems;
- narrow the target segment further;
- pivot if creators will not delegate or pay.

Do not begin a paid plugin marketplace or consumer game store during this period.

19. Lean Canvas

Canvas block	ThreeNative hypothesis
Problem	AI-generated 3D games are often disposable; creators cannot trust long autonomous changes; native/mobile shipping is painful; common game systems are regenerated ad hoc; closed tools create lock-in.
Customer segments	Beachhead: JS-literate AI power users with recent 3D game attempts. Expansion: visual creators who prefer directing over operating an engine. Early adopters: creators with an existing fragile prototype.
Unique value proposition	Direct a game from references and intent; receive verified native builds and real Three.js source you own.
High-level concept	"An AI game director's workshop: Three.js-native, batteries included, evidence attached."
Solution	Studio intent contract, autonomous plan/build loop, reusable systems, playable preview, verification receipts, checkpoints, source export, managed native builds.
Channels	Open-source framework, Three.js communities, AI coding communities, case-study videos, game jams, existing-project migration, creator referrals.
Revenue streams	Studio subscriptions, usage overage, higher-tier device/build services, later paid plugins, publishing services, and marketplace revenue share.
Cost structure	Models, orchestration compute, native build/device farm, storage/bandwidth, asset generation, support, framework engineering, later moderation and payments.
Key metrics	Accepted verified iterations, first native build, week-3/week-8 retention, cost per accepted milestone, verification false positives, paid continuation.
Unfair advantage hypothesis	Upstream Three.js model familiarity + native GPU runtime + AI-legible game framework + verification protocol + open escape hatches + accumulating project/plugin evidence. This advantage must be built; it is not yet guaranteed.

20. Decision scorecard

Review this scorecard monthly during discovery and quarterly after launch.

Hypothesis	Current confidence	Evidence needed next	Kill / change signal
AI-first game directors are a reachable segment	Medium	15-20 recent-behavior interviews, paid pilots	Interviews are mostly hypothetical or no one provides a real project
Users will delegate implementation	Medium-low	Observe plan approval and autonomous sessions	Most users demand continuous scene-level control
Verification drives willingness to pay	Medium	A/B solution interviews and paid pilot feedback	Users ignore receipts and manually retest everything
Native Android/desktop delivery is differentiated	Medium-high	Paid selection and native install behavior	Users are satisfied with browser export and will not pay more
Three.js/source openness reduces lock-in	Medium	Objection analysis and export usage	Few users care, while framework complexity slows delivery
Reusable systems reduce cost and defects	High conceptually	Controlled implementation comparisons	Systems are frequently bypassed or make output too generic
Subscription + credits is acceptable	Low	Paid pricing tests and surprise/refund tracking	Spend feels unpredictable or users prefer fixed milestone pricing
Open framework creates Studio acquisition	Medium-low	Attribution and conversion data	OSS users remain separate and do not need managed Studio
Plugin market can expand capability	Low, later	External plugin adoption in paid projects	Plugins create more incompatibility/support cost than value
Curated game marketplace has player demand	Very low, later	Limited showcases, bundles, repeat-player data	Players reject catalog quality or creators already prefer established stores

21. Final recommendation

ThreeNative has a coherent opportunity when described narrowly:

It is not a game engine trying to outbuild Unity. It is an AI-native creation system that uses the renderer and programming model AI already knows, adds the missing game and native-runtime layers, and makes delegated work verifiable.

The open-source framework is strategically important, but **Studio is the product to validate first**. The product should begin with technically capable AI-first creators, a constrained 3D game wedge, real native delivery, and an unusually strong proof-and-rollback loop.

The plugin ecosystem is a logical second-order advantage because it lets agents install reliable capabilities instead of regenerating them. A curated game marketplace is a compelling long-term flywheel, but it should be earned through creator retention, quality finished games, and evidence of player demand.

The immediate company question is not whether ThreeNative can eventually support every platform or game system. It is:

Will ten real creators repeatedly pay to direct a game while ThreeNative performs and proves most of the implementation?

Answer that with paid behavior. Everything else becomes easier to sequence.

Appendix A. Sample intent contract

Project identity

- Working title:
- One-sentence fantasy:
- Target player:
- Intended session length:
- Reference images/videos:

Experience goals

- The player should feel:
- The player repeatedly does:
- The most important moment is:
- The game is successful when:

Locked constraints

- Must include:
- Must not include:
- Visual elements to preserve:
- Mechanics not to alter without approval:
- Content/safety boundaries:

Technical contract

- Target platforms:
- Input modes:
- Minimum frame rate / frame-time budget:
- Memory/download constraints:
- Offline/online requirements:
- Required accessibility:

Current milestone

- Goal:
- Scope:
- Acceptance criteria:
- Automated play route:
- Required screenshots:
- Native devices/builds:
- Maximum credit budget:
- Human approval points:

Appendix B. Verification receipt schema

```
milestone: forest-combat-loop-v2
checkpoint_before: 4f92a1
checkpoint_after: 5c7bd8
intent:
  goal: "Combat should feel faster without changing the hostile forest tone"
  locked_constraints:
    - "Do not change player silhouette"
    - "Maintain 60 fps target on reference Android device"
changes:
  source_files: 8
  assets: 2
  systems:
    - character-controller
    - melee-combat
verification:
  unit_tests: {passed: 24, failed: 0}
  play_routes:
    - name: tutorial-to-first-enemy
      result: passed
    - name: death-and-restart
      result: passed
  visual_captures:
    - frame: combat-entry
  artifact: combat-entry.png
  performance:
    device: reference-android
    median_frame_ms: 14.8
    p95_frame_ms: 16.1
  native_build:
    artifact: forest-runner-v2.apk
    launch_test: passed
uncertainty:
  inferred:
    - "Difficulty remains appropriate after the first encounter"
  not_evaluated:
    - "Long-session thermal behavior"
creator_decision: pending
rollback_to: 4f92a1
```

Appendix C. Research sources and caveats

The external numbers in this report are snapshots used to frame hypotheses, not market-size estimates. Three.js npm downloads may include CI, mirrors, repeated installs, and transitive use. Developer and game-industry surveys have sampling limitations. Competitor capabilities and pricing can change. ThreeNative's customer and willingness-to-pay conclusions must therefore come from direct research and product behavior.

Core product methodology

- Dan Olsen, *The Lean Product Playbook*: <https://dan-olsen.com/book/>
- Wiley Online Library edition and chapter index: <https://onlinelibrary.wiley.com/doi/book/10.1002/9781119154822>
- Ash Maurya, *Lean Canvas* and *Running Lean*: <https://ashmaurya.com/> and <https://app.leanstack.com/>
- Sean Ellis Product-Market Fit Survey: <https://pmfsurvey.com/>
- First Round Review, Superhuman product-market-fit case study: <https://review.firstround.com/how-superhuman-built-an-engine-to-find-product-market-fit/>
- Nielsen Norman Group, qualitative user testing: <https://www.nngroup.com/consulting/user-testing/>

Market and behavior evidence

- Stack Overflow 2025 Developer Survey, AI: <https://survey.stackoverflow.co/2025/ai>
- Game Developers Conference 2026 State of the Game Industry: <https://gdconf.com/article/gdc-2026-state-of-the-game-industry-reveals-impact-of-layoffs-generative-ai-and-more/>
- npm, three package: <https://www.npmjs.com/package/three>

Product and competitor references

- Rosebud AI Game Creator: <https://rosebud.ai/ai-game-creator>
- Roblox, "Roblox Studio Is Going Agentic": <https://about.roblox.com/newsroom/2026/04/roblox-studio-going-agentic>
- GDevelop: <https://gdevelop.io/>
- PlayCanvas: <https://playcanvas.com/>
- Construct: <https://www.construct.net/en>

Ecosystem and marketplace references

- Unity Asset Store publisher program: <https://assetstore.unity.com/publishing/publish-and-sell-assets>
- Valve Steam Direct: <https://partner.steamgames.com/steamdirect>
- itch.io marketplace model: <https://itch.io/docs/general/about>

All web sources were accessed on August 27, 2026 unless otherwise indicated in the footnotes.